

Day 4 — Day 3 to Day 4 RAG Correlation + Code Walkthrough

This PDF is intentionally code-focused and demo-ready. It connects the Day 3 notebook workflow to the Day 4 project code so the instructor can point to exact snippets, explain the equivalent logic, and run the same idea live in VS Code.

The goal is not to summarize frameworks. The goal is to show that the underlying architecture is the same: ingest documents, split them, embed them, store vectors, retrieve relevant chunks, and pass them to an LLM.

Page 1 — The real message: same RAG architecture, different tooling

Use this as the opening explanation before touching code. The notebook and the project are not different ideas; they are two implementations of the same architecture.

Day 3 notebook flow:

SimpleDirectoryReader -> Document -> SentenceSplitter -> HuggingFaceEmbedding ->

LanceDBVectorStore -> VectorStoreIndex -> query_engine

Day 4 project flow:

src/pdf_parser.py -> src/chunker.py -> SentenceTransformerEmbeddingFunction -> Chroma collection

-> src/vector_store.py.query() -> src/rag.py

What to say out loud: “The notebook used a high-level framework. The project does the same steps manually in Python. The architecture is the same, but the abstraction layer is different.”

This is the teaching point students need to understand before any code highlighting begins.

Page 2 — Day 3 notebook equivalent: ingestion and document preparation

In the notebook, you likely saw code similar to this pattern from LlamaIndex:

```
from llama_index.core import SimpleDirectoryReader, Document
documents = SimpleDirectoryReader("data/").load_data()
for doc in documents:
    print(doc.text[:200])
```

This corresponds directly to the project code in `src/main.py` and `src/pdf_parser.py`, where the system extracts text from PDFs and prepares it for chunking.

```
pages = extract_pages(pdf_path)
chunks = chunk_text(pages, source=source, chunk_size=300, overlap=50)
store.add_chunks(chunks)
```

What to say out loud: “The notebook loaded documents into a document abstraction. Our project loads PDF pages and converts them into chunk objects. Same step—different representation.”

Page 3 — Day 3 notebook equivalent: chunking with SentenceSplitter

In the notebook, the chunking step was likely done by LlamaIndex's SentenceSplitter. That is the same conceptual role as src/chunker.py, which splits raw text into retrievable units.

```
from llama_index.core.node_parser import SentenceSplitter
splitter = SentenceSplitter(chunk_size=300, chunk_overlap=50)
nodes = splitter.get_nodes_from_documents(documents)
```

In Day 4, the equivalent code is here:

```
for page in pages:
    words = page["text"].split()
    end = start + chunk_size
    chunk_words = words[start:end]
    chunk_str = " ".join(chunk_words)
    chunks.append({
        "text": chunk_str,
        "page": page["page"],
        "chunk_id": chunk_id,
    })
    start = end - overlap
```

What to say out loud: “SentenceSplitter in the notebook is just a framework wrapper around the same operation we are implementing manually in src/chunker.py. Both create smaller, overlapping chunks so no fact is prematurely cut off.”

Page 4 — Day 3 notebook equivalent: embeddings with HuggingFaceEmbedding

In the notebook, the embedding step was probably done with HuggingFaceEmbedding and an IngestionPipeline. The Day 4 version is explicit in src/vector_store.py.

```
from llama_index.embeddings.huggingface import HuggingFaceEmbedding
embed_model = HuggingFaceEmbedding(model_name="BAAI/bge-small-en-v1.5")
pipeline = IngestionPipeline(transformations=[SentenceSplitter(...), embed_model],
vector_store=vector_store)
```

Equivalent code in the project:

```
self.embedding_fn = embedding_functions.SentenceTransformerEmbeddingFunction(
model_name="all-MiniLM-L6-v2"
)
self.collection = self.client.get_or_create_collection(
name=collection_name,
embedding_function=self.embedding_fn,
metadata={"hnsw:space": "cosine"},
)
```

What to say out loud: “Both systems convert each chunk into a vector. In the notebook, the framework hides the details. In the project, we are doing the same step explicitly and storing the result in Chroma.”

Page 5 — Day 3 notebook equivalent: vector DB storage with LanceDB

In the notebook, the vector database was likely LanceDB. In the project, the equivalent is ChromaDB. The conceptual job is identical: persist the vectors so they can be searched later.

```
from llama_index.vector_stores.lancedb import LanceDBVectorStore
vector_store = LanceDBVectorStore(uri="./lancedb_data", table_name="docs")
```

Equivalent project code:

```
self.client = chromadb.PersistentClient(path=persist_dir)
self.collection = self.client.get_or_create_collection(
    name=collection_name,
    embedding_function=self.embedding_fn,
    metadata={"hnsw:space": "cosine"},
)
```

What to say out loud: “This is the storage layer. The database is where the vectorized document knowledge lives. The notebook used LanceDB; the project uses Chroma. Same job, different backend.”

Page 6 — Day 3 notebook equivalent: query engine and retrieval

This is the most important connection point. In the notebook, the framework created a query engine that retrieved top matches and sent them to the LLM. In the project, the retrieval logic is written explicitly in `src/vector_store.py` and orchestrated in `src/rag.py`.

```
index = VectorStoreIndex.from_vector_store(vector_store=vector_store, embed_model=embed_model)
query_engine = index.as_query_engine(response_mode="tree_summarize")
response = query_engine.query(question)
```

Equivalent project code:

```
results = self.collection.query(query_texts=[question], n_results=fetch_k, where=where)
for rank, (text, meta, dist) in enumerate(
    zip(results["documents"][0], results["metadatas"][0], results["distances"][0]), start=1
):
    candidates.append({
        "text": text,
        "page": meta["page"],
        "source": meta["source"],
        "distance": dist,
        "confidence": 1 - (dist / 2),
    })
```

What to say out loud: “This is the exact retrieval step. The notebook had a query engine abstraction; the project exposes the same behavior in plain Python. We are still asking the vector store for the nearest relevant passages.”

Page 7 — Day 3 notebook equivalent: LLM answer generation and model provider layer

In the notebook, the LLM was likely called through a wrapper or an inference API. In the project, the LLM is selected centrally in `src/llm_config.py` and the actual orchestration sits in `src/rag.py`.

```
llm = HuggingFaceInferenceAPI(model_name="...")
response = query_engine.query(question)
```

Equivalent project code:

```
raw_active_provider = (os.environ.get("ACTIVE_PROVIDER") or "").strip().lower()
if provider == "openrouter":
    return LLMSettings(
        name="openrouter",
        model=os.environ.get("OPENROUTER_MODEL") or "openai/gpt-oss-20b:free",
        api_key=os.environ.get("OPENROUTER_API_KEY"),
    )
SYSTEM_PROMPT = """Document-first policy..."""
```

What to say out loud: “The notebook simplified the model call. Our project makes the provider and model selection explicit, which is what makes it easier to switch between OpenRouter, Gemini, and Hugging Face in real usage.”

Page 8 — The conceptual continuity to repeat in class

Use this exact explanation live:

“Day 3 was a framework-based proof of concept. Day 4 is the same RAG architecture implemented as a real Python app.”

“The notebook used LlamaIndex, LanceDB, and a simplified query engine. The project uses custom parsing, chunking, Chroma, and explicit retrieval logic.”

“The logic is the same even though the APIs are different.”

Then say: “We are not building a different concept. We are building the same concept with different tools and more control.”

This is the line to anchor the teaching story and help students see continuity instead of a jump to a new project.

Page 9 — PDF ingestion: where the indexing starts

Show this live in `src/main.py`. This is the counterpart of the notebook's ingestion workflow. The important idea is: read a PDF, split it into chunks, insert into vector DB.

```
pages = extract_pages(pdf_path)
chunks = chunk_text(pages, source=source, chunk_size=300, overlap=50)
store.add_chunks(chunks)
```

What to say out loud: "This is where the indexing begins. We are turning raw PDF pages into chunk objects that can be embedded and searched."

Point to: `src/main.py`, `index_pdf()`

Page 2 — Chunking: this is where "retrievable units" are created

Show this live in `src/chunker.py`. This is the exact place where a long page gets broken into smaller chunks with overlap.

```
for page in pages:
    words = page["text"].split()
    end = start + chunk_size
    chunk_words = words[start:end]
    chunk_str = " ".join(chunk_words)
    chunks.append({
        "text": chunk_str,
        "page": page["page"],
        "chunk_id": chunk_id,
    })
    start = end - overlap
```

What to say out loud: “Notice the overlap. If we only split without overlap, an important fact can be cut across boundary lines. Overlap reduces that risk.”

This is the code-level equivalent of `SentenceSplitter` in the notebook.

Page 3 — Embeddings: where the vectors are created

Show this live in `src/vector_store.py`. This is the exact place where Chroma is configured and the embedding model is declared.

```
self.embedding_fn = embedding_functions.SentenceTransformerEmbeddingFunction(
    model_name="all-MiniLM-L6-v2"
)
self.collection = self.client.get_or_create_collection(
    name=collection_name,
    embedding_function=self.embedding_fn,
    metadata={"hnsw:space": "cosine"},
)
```

What to say out loud: “This is the exact embedding step. Each chunk is converted to a vector and stored in the database.”

This replaces the notebook’s `HuggingFaceEmbedding` / `LlamaIndex` abstraction with explicit embedding logic in Python.

Page 4 — Storage: how Chroma stores document metadata

Show this live in `src/vector_store.py`, `add_chunks()`. This is where metadata is attached to each chunk. That metadata matters later for page references, source tracking, and citation.

```
self.collection.add(
    ids=[f"{c['source']}::{c['chunk_id']}" for c in chunks],
    documents=[c["text"] for c in chunks],
    metadatas=[
        {
            "page": c["page"],
            "source": c["source"],
            "chunk_id": c["chunk_id"],
        }
        for c in chunks
    ],
)
```

What to say out loud: “This is not just raw text storage. We are storing file source, page number, and chunk identity so we can cite the evidence later.”

Page 5 — Retrieval: this is the real vector-search step

Show this live in `src/vector_store.py`, `query_with_candidates()`. This is the exact analog of the notebook's query engine.

```
results = self.collection.query(query_texts=[question], n_results=fetch_k, where=where)
for rank, (text, meta, dist) in enumerate(
    zip(results["documents"][0], results["metadatas"][0], results["distances"][0]), start=1
):
    candidates.append({
        "text": text,
        "page": meta["page"],
        "source": meta["source"],
        "distance": dist,
        "confidence": 1 - (dist / 2),
        "rank": rank,
    })
```

What to say out loud: “This is the retrieval step. We ask the vector DB for the most similar chunks, rank them by distance, and compute a confidence score. This is exactly what the query engine does under the hood in the notebook.”

Page 6 — Prompting + tool use: where the model decides whether to call a tool

Show this live in `src/rag.py`. This is the most important conceptual link to the notebook's query engine and tool-calling idea.

```
SYSTEM_PROMPT = """You are a helpful research assistant and teacher with two tools:
- search_documents: searches the user's uploaded PDF documents.
- web_search: searches the live internet.
...
"""
TOOLS_SCHEMA = [
{"name": "search_documents", ...},
{"name": "web_search", ...},
]
```

What to say out loud: “This is the policy layer. The model is told: document first, web as fallback, do not loop endlessly, and do not invent facts. The notebook may have hidden this in the framework, but here it is explicit in code.”

Page 7 — The confidence gate: code-level safeguard against bad retrieval

Show this live in `src/rag.py`. This is where we stop an answer from being based on weak document match quality.

```
if gate_state["web_called"] or not web_enabled or confidence >= threshold:
    return "", []
results = _web_search(query_text, max_results=5, allowed_domains=allowed_domains,
                      blocked_domains=blocked_domains)
gate_state["web_called"] = True
gate_state["auto_triggered"] = True
```

What to say out loud: “This is the fix for the bad behavior we saw earlier. The model is not allowed to rely only on weak document similarity if the confidence is low. The code forces a web fallback instead of letting the system guess.”

Page 8 — Document-first policy: the exact safeguard to teach students

This is the key rule that should be emphasized. Show the SYSTEM_PROMPT block from src/rag.py and explain that it sets behavior, not just response style.

```
Document-first policy: always search_documents first for questions that could be answered from the indexed PDFs.
```

```
Never use web_search before search_documents when the document corpus could plausibly answer the question.
```

```
Use web_search only as a fallback for missing facts, current information, dates, names, places, events, or if the document search returns no useful result.
```

What to say out loud: “This is the entire principle behind the app. We do not let the web bypass the document corpus when the document corpus could answer the question.”

Page 9 — Provider switching: the model layer is configurable, not hard-coded

Show this live in `src/llm_config.py`. This is the exact point where the app decides which provider is active and which model is chosen.

```
raw_active_provider = (os.environ.get("ACTIVE_PROVIDER") or "").strip().lower()
if raw_active_provider in {"groq", "claude", "huggingface", "gemini", "openrouter"}:
    provider = raw_active_provider
    if provider == "openrouter":
        return LLMSettings(
            name="openrouter",
            model=os.environ.get("OPENROUTER_MODEL") or "openai/gpt-oss-20b:free",
            api_key=os.environ.get("OPENROUTER_API_KEY"),
        )
```

What to say out loud: “This is where the app becomes portable. We are not mixing logic for every provider in the RAG flow itself. The provider selection is centralized in config.”

Page 10 — Live demo checklist for the instructor

- Open `src/main.py` and show `index_pdf()`
- Open `src/chunker.py` and highlight the overlap logic
- Open `src/vector_store.py` and highlight `SentenceTransformerEmbeddingFunction` and `collection.add()`
- Open `src/vector_store.py` and show `query_with_candidates()`
- Open `src/rag.py` and highlight `SYSTEM_PROMPT` and the document-first policy
- Open `src/rag.py` and show the confidence gate
- Open `src/app.py` and show the Streamlit UI controls

This is not a conceptual summary slide. This is the exact execution order to follow live so students see the full RAG pipeline in real code.